

# 1. Count-Based Bigram

- Build character vocabulary and index maps.
- Add start/end token to every word.
- Count adjacent character pairs.
- $N[i, j]$  stores a count, not probability.



| next → | . | a | b | c |
|--------|---|---|---|---|
| cur .  | 0 | 2 | 0 | 0 |
| cur a  | 1 | 1 | 3 | 0 |
| cur b  | 1 | 0 | 0 | 1 |

$N[a, b] = 3$

The character **b** appeared after **a** three times.

emma → . e m m a . → pairs

```
# build vocabulary
chars =
sorted(list(set(''.join(words))))
stoi = {s:i+1 for i,s in
enumerate(chars)}
stoi['.'] = 0
itos = {i:s for s,i in stoi.items()}

# count bigrams
N = torch.zeros((27, 27),
dtype=torch.int32)

for w in words:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs,
chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        N[ix1, ix2] += 1
```

## 2. Probability Matrix

makemore part 1 / 02

- Add smoothing before normalization.
- Turn each count row into probabilities.
- Each row is  $P(\text{next char} \mid \text{current char})$ .
- `keepdim=True` preserves row shape.



[1, 3, 2] +1 [2, 4, 3] /9



**One row becomes one distribution.**

After normalization, every row sums to 1.

```
# add smoothing
P = (N + 1).float()

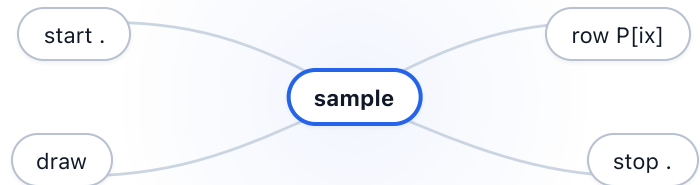
# normalize rows
P = P / P.sum(1, keepdim=True)

# meaning
# P[i, j] = probability of char j
# after current char i

P.shape
P.sum(1)
```

### 3. Sampling Names

- Start from dot token.
- Use current row as probabilities.
- Sample one next-character index.
- Stop when sampled token is dot.



#### Sampling is iterative.

The sampled character becomes the next input row.

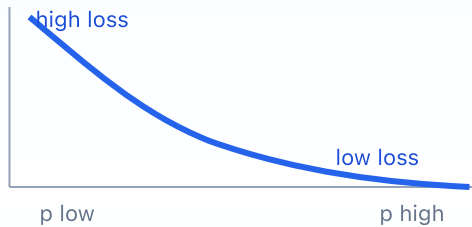
```
g =
torch.Generator().manual_seed(2147483647)

for i in range(5):
    out = []
    ix = 0
    while True:
        p = P[ix]
        ix = torch.multinomial(p, 1,
replacement=True,

generator=g).item()
        out.append(itos[ix])
        if ix == 0:
            break
    print(''.join(out))
```

## 4. Negative Log Likelihood

- Look up probability of the true next char.
- High probability gives small penalty.
- Low probability gives large penalty.
- Average NLL is the loss.



$$\text{NLL} = -\log(p_{\text{true}})$$

If the true next char gets low probability, the penalty is large.

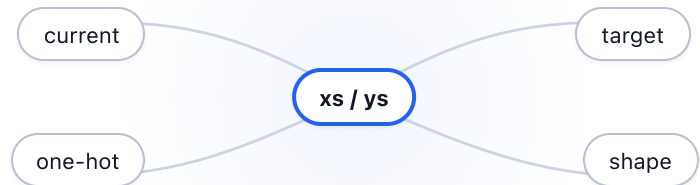
```
log_likelihood = 0.0
n = 0

for w in words:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs,
                        chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        prob = P[ix1, ix2]
        log_likelihood +=
            torch.log(prob)
        n += 1

loss = -log_likelihood / n
```

## 5. Training Data and One-Hot

- Convert bigrams into input-target pairs.
- xs is current character index.
- ys is next character index.
- One-hot turns indices into vectors.



### One-hot is a selector.

It keeps identity while making matrix multiplication possible.

```
xs, ys = [], []

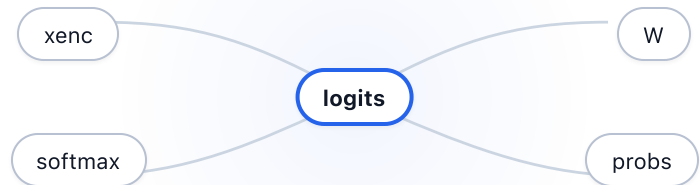
for w in words:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs,
                        chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        xs.append(ix1)
        ys.append(ix2)

xs = torch.tensor(xs)
ys = torch.tensor(ys)
xenc = F.one_hot(xs,
                 num_classes=27).float()
```

## 6. Neural Bigram Forward Pass

makemore part 1 / 06

- $W$  starts as random trainable logits.
- $x_{enc} @ W$  selects a row of  $W$ .
- $\exp$  turns logits into positive counts.
- Normalize to get probabilities.



0 1 0 @ W = W[row 1]

logit .

logit a

logit b

### $x_{enc} @ W$ selects rows.

The selected row contains logits for the next character.

```
g =  
torch.Generator().manual_seed(2147483647)  
W = torch.randn((27, 27),  
generator=g,  
requires_grad=True)
```

```
logits = xenc @ W  
counts = logits.exp()  
probs = counts / counts.sum(1,  
keepdim=True)
```

```
loss = -  
probs[torch.arange(xs.nelement(),  
ys)].log().mean()
```

# 7. Gradient Descent and Comparison

- Backward computes how  $W$  affects loss.
- Update  $W$  against the gradient direction.
- Count-based  $P$  is measured from data.
- Neural  $P$  is learned by optimization.



forward

loss

backward

update  $W$

## Training changes $W$ .

The goal is to lower average NLL over the dataset.

```
for k in range(100):
    logits = xenc @ W
    counts = logits.exp()
    probs = counts / counts.sum(1,
keepdim=True)
    loss = -
    probs[torch.arange(xs.nelement()),
ys].log().mean()

    W.grad = None
    loss.backward()
    W.data += -50 * W.grad

# count model: N -> P
# neural model: W -> logits -> probs
```